Memoria Práctica 1 Sistemas Operativos

Carlos Méndez Barrasa y Ana Olsson Rodríguez

Grupo 2262

Índice

1. Requisitos cumplidos y no cumplidos..... 2
1.1. Sistema multiproceso.....2
1.2. Minero multihilo.....2
1.3. Registrador del sistema.....3
1.4. Pruebas y razonamiento.....5
2. Pruebas realizadas..... 7
3. Respuesta a las cuestiones del enunciado..... 7

1. Requisitos cumplidos y no cumplidos

En esta práctica se pedía programar la primera entrega del proyecto incremental denominado "Miner Rush", donde un conjunto de procesos (mineros) tendrán que resolver una prueba de esfuerzo (POW), en este caso, a través de una función hash, de manera concurrente. Sólo el más rápido de ellos en encontrarla será recompensado. Los objetivos de dicha entrega inicial eran los siguientes:

1.1. Sistema multiproceso

Se pide un sistema que se ejecute de la siguiente manera: `./miner <TARGET_INT> <ROUNDS> <N_THREADS>`, donde `miner` es el nombre del ejecutable, `<TARGET_INT>` es el objetivo del problema inicial, `<ROUNDS>` es el número de rondas que se van a realizar y `<N_THREADS>` es el número de hilos que se van a utilizar.

El proceso principal, `Minero`, deberá crear un nuevo proceso usando fork: el proceso `Registrador`. Además, `Minero` y `Registrador` tendrán que indicar por pantalla el código de salida a través de las cadenas `"Miner exited with status %d"` / `"Logger exited with status %d"` / `"Miner exited unexpectedly"` / `"Logger exited unexpectedly"`, según los procesos terminen normalmente o no.

Se puede comprobar que hemos cumplido este requisito a través de la ejecución de `miner`, que acepta los argumentos solicitados y muestra por pantalla los mensajes pertinentes.

1.2. Minero multihilo

En este apartado se solicita que cada Minero sea un proceso encargado de resolver la POW usando múltiples hilos en paralelo que reciban el número de rondas de minado que debe realizar, el número de hilos que debe utilizar, y el objetivo del problema inicial que debe resolver.

En cada ronda el proceso `Minero`:

- Divide el espacio de búsqueda entre tantos hilos como se haya especificado, de manera que realicen búsquedas independientes para realizar la tarea
- Crea los hilos, y espera a que terminen.
- Cuando un hilo encuentra la solución, terminan todos los hilos y se devuelve la solución al hilo principal del proceso `Minero`.
- El proceso `Minero` fija como siguiente objetivo la solución obtenida en esta ronda.

Además, el proceso sale con código `EXIT_FAILURE` si se produce algún error, y con `EXIT_SUCCESS` si todo fue correctamente.

En nuestro código, el proceso `Minero` es `parentMiner` que despliega sus hilos a través de `minerRound`, que a su vez buscarán el POW con `minerWorker`. Este requisito también se ha

cumplido tal y como puede verse ejecutando el código adjunto a esta Memoria, que, siguiendo la recomendación de la nota, imprime la solución tras cada ronda de minado. A continuación se muestra un ejemplo (se aprecia, además, el requisito anterior):

```
● (base) anaol@AnaOlsson:/mnt/c/UAM/2_carrera/2_cuatri/SOPER/SOPA/Ejercicio_13$ ./miner 0 5 3
  Solution accepted: 00000000 --> 8916551
  Solution accepted: 08916551 --> 5970796
  Solution accepted: 05970796 --> 937352
  Solution accepted: 00937352 --> 1835314
  Solution accepted: 01835314 --> 9331340
  Miner exited with status 0
  Logger exited with status 0
```

1.3. Registrador del sistema

Este requisito consta de que el `Registrador` del sistema:

- El proceso `Registrador` será el encargado de registrar en un fichero las soluciones encontradas por el proceso `Minero`.
- Para ello, `Minero` enviará un mensaje a través de una tubería al `Registrador`, indicándole el objetivo y la solución obtenida.
- `Registrador` escribirá en el fichero la información pertinente de registro. Cuando haya escrito el resultado en el fichero, enviará un mensaje de confirmación a través de tubería a `Minero`, para que continúe con la siguiente ronda.
- El proceso `Minero` será el que determine cuándo ha completado la última ronda, y mandará una orden por la tubería para la finalización del proceso Registrador al que esperará (por ejemplo una solución -1 encontrada). `Registrador` terminará su ejecución saliendo con código `EXIT_FAILURE` si se produce algún error, y con código `EXIT_SUCCESS` si todo fue correctamente.

Mientras la tubería que lo conecta con `Minero` se mantenga abierta, lee la información recibida, y los escribe usando la función `dprintf` en un fichero cuyo nombre dependa del PID de su proceso padre. Además, el fichero generado deberá tener una estructura concreta, definida en el enunciado de la práctica.

Para confirmar que hemos completado este requisito podemos ver el código y el archivo generado, a continuación adjuntamos un fragmento de un archivo generado mediante `./miner 0 10 3` (Hemos escogido 10 rondas ya que la probabilidad de que se rechace una solución es del 10%):

```
Id:       1
Winner:   505756
Target:   00000000
Solution: 8916551 (validated)
Votes:    1/1
Wallets:  505756:1
```

```
Id:       2
Winner:   505756
Target:   08916551
Solution: 5970796 (validated)
Votes:    2/2
Wallets:  505756:2

Id:       3
Winner:   505756
Target:   05970796
Solution: 937352 (validated)
Votes:    3/3
Wallets:  505756:3

Id:       4
Winner:   505756
Target:   00937352
Solution: 1835314 (validated)
Votes:    4/4
Wallets:  505756:4

Id:       5
Winner:   505756
Target:   01835314
Solution: 9331340 (validated)
Votes:    5/5
Wallets:  505756:5

Id:       6
Winner:   505756
Target:   09331340
Solution: 2986686 (validated)
Votes:    6/6
Wallets:  505756:6

Id:       7
Winner:   505756
Target:   02986686
Solution: 7299854 (validated)
Votes:    7/7
Wallets:  505756:7
```

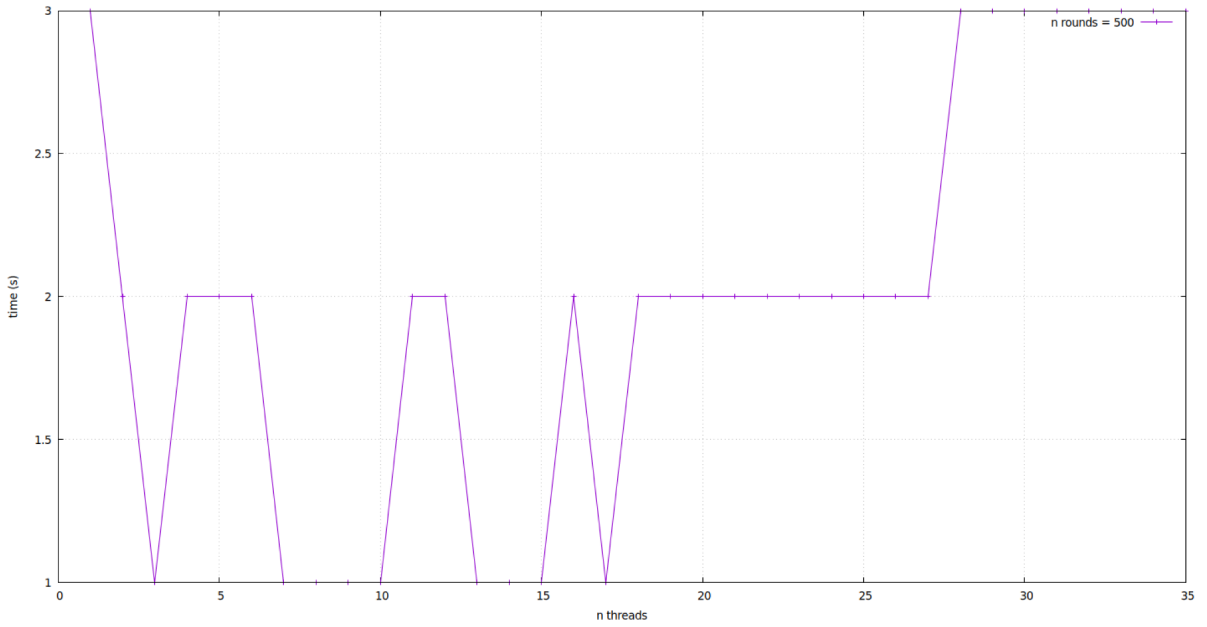
```
Id:       8
Winner:   505756
Target:   07299854
Solution: 3981310 (validated)
Votes:    8/8
Wallets:  505756:8

Id:       9
Winner:   505756
Target:   03981310
Solution: 8021305 (rejected)
Votes:    9/9
Wallets:  505756:9

Id:       10
Winner:   505756
Target:   08021305
Solution: 4673876 (validated)
Votes:    10/10
Wallets:  505756:10
```

1.4. Pruebas y razonamiento

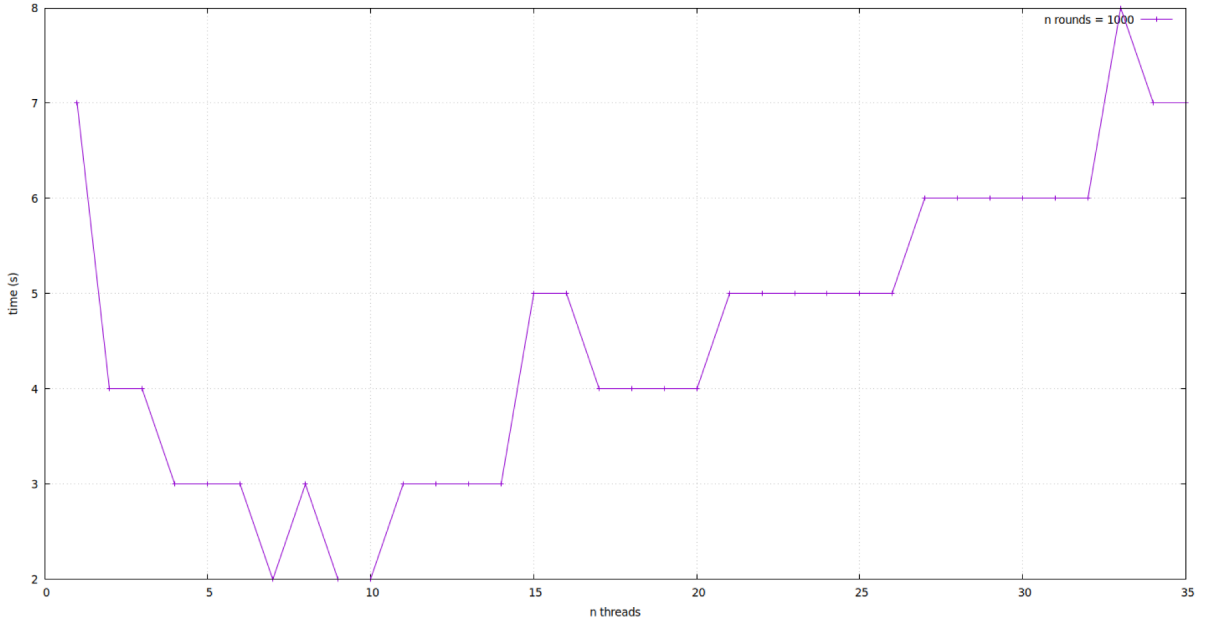
Para terminar, se pide que hagamos pruebas con diferente número de rondas y diferente número de hilos, tomando tiempos de ejecución. Para poder hacer comparaciones hemos decidido crear un bashfile que genere tiempos de ejecución de miner para el mismo target y número de rondas pero distinto número de hilos a través de la llamada iterada a un miner modificado que temporiza su ejecución y lo registra en un fichero “tiempos.data”.

A continuación, mostramos diversas gráficas de los tiempos de ejecución en función del número de hilos para diversos números de rondas.

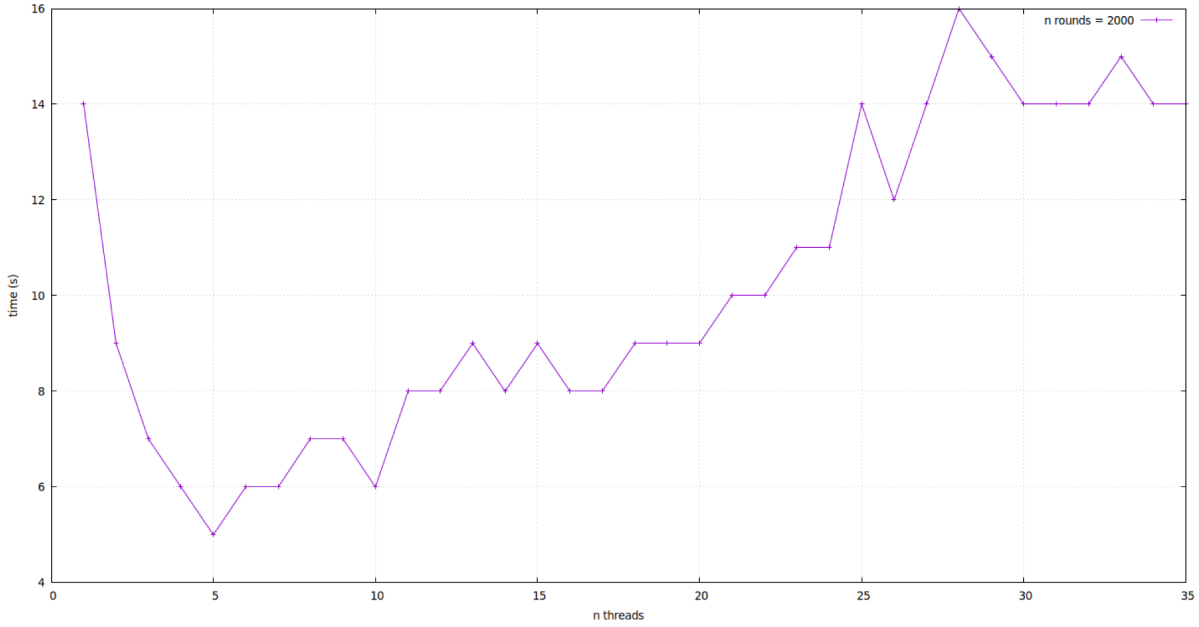
500 rondas:



1000 rondas:



2000 rondas:



2. Pruebas realizadas

Las pruebas que hemos realizado para comprobar el funcionamiento del programa son la ejecución del mismo con diversos parámetros, ya sea a través de la llamada explícita a `miner` por terminal o a través del bashfile.

Varios ejemplos de ejecución pueden verse en los anteriores apartados, de cara a los casos límites hemos probado a ejecutarlo con entradas que no tuvieran sentido tales como número negativos o probando su robustez a través de la ejecución con parámetros exagerados. Comprobamos que funciona bien seleccionando 1000 rondas y 100 hilos (aunque tarda mucho en terminar) y de cara a los números negativos obtenemos:

```
⊗ (base) anaol@AnaOlsson:/mnt/c/UAM/2_carrera/2_cuatri/SOPER/SOPA/Ejercicio_13$ ./miner -1 -1 -1
  Invalid arguments
  : Success
⊗ (base) anaol@AnaOlsson:/mnt/c/UAM/2_carrera/2_cuatri/SOPER/SOPA/Ejercicio_13$ ./miner 1 -5 3
  Invalid arguments
  : Success
⊗ (base) anaol@AnaOlsson:/mnt/c/UAM/2_carrera/2_cuatri/SOPER/SOPA/Ejercicio_13$ ./miner 1 5 -3
  Invalid arguments
  : Success
⊗ (base) anaol@AnaOlsson:/mnt/c/UAM/2_carrera/2_cuatri/SOPER/SOPA/Ejercicio_13$ ./miner -1 5 3
  Invalid arguments
  : Success
```

Además, confirmamos que no hubiera errores de memoria a través de valgrind:

```
(base) anaol@AnaOlsson:/mnt/c/UAM/2_carrera/2_cuatri/SOPER/SOPA/Ejercicio_13$ make valgrind
valgrind -s --show-leak-kinds=all --track-origins=yes --leak-check=full ./miner 0 10 10 > valgrind.data
==2729147== Memcheck, a memory error detector
==2729147== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==2729147== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==2729147== Command: ./miner 0 10 10
==2729147==
==2729189==
==2729189== HEAP SUMMARY:
==2729189==     in use at exit: 0 bytes in 0 blocks
==2729189==   total heap usage: 1 allocs, 1 frees, 4,096 bytes allocated
==2729189==
==2729189== All heap blocks were freed -- no leaks are possible
==2729189==
==2729189== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
==2729147==
==2729147== HEAP SUMMARY:
==2729147==     in use at exit: 0 bytes in 0 blocks
==2729147==   total heap usage: 87 allocs, 87 frees, 28,072 bytes allocated
==2729147==
==2729147== All heap blocks were freed -- no leaks are possible
==2729147==
==2729147== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

3. Respuesta a las cuestiones del enunciado

A continuación, incluimos las cuestiones junto con sus respuestas.

- ¿Qué conclusiones se pueden tomar respecto al número óptimo de hilos?

Observando las gráficas del apartado 1.4. vemos que el rendimiento óptimo ronda los 5-16 hilos, el ordenador sobre el que se calcularon posee 16 cores por lo que el número de hilos óptimo es igual o inferior a número de cores del procesador sobre el que se ejecute.